

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY



CHUYÊN ĐỀ HỆ THỐNG PHÂN TÁN (22_4)

LAB01:

SES

Advisor(s): Huỳnh Thụy Bảo Trân
Dũng Trần Trung
Student(s): Trần Hùng Anh

Ho Chi Minh City, 10/2025

Mục lục

I	Giới thiệu nhóm và tiến độ công việc	3
1	Thông tin sinh viên	3
2	Khó khăn và giải pháp	3
2.1	Thread Synchronization	3
2.2	Message Ordering	3
2.3	Debugging với Multiple Processes	3
3	Link tài liệu	3
II	Báo cáo bài tập	4
4	Tổng quan đồ án	4
4.1	Mục tiêu	4
4.2	Yêu cầu đề bài	4
5	Cơ sở lý thuyết	4
5.1	Thuật toán SES	4
5.1.1	Vector Clock	4
5.1.2	Điều kiện delivery	4
5.1.3	Thuật toán	5
6	Thiết kế hệ thống	5
6.1	Kiến trúc tổng quan	5
6.2	Cấu trúc Process	5
6.3	Cấu trúc dữ liệu	6
6.3.1	VectorClock Class	6
6.3.2	Message Class	7
6.3.3	SESPProcess Class	7
7	Cài đặt	7
7.1	Ngôn ngữ và công cụ	7
7.2	Cấu trúc dự án	8
7.3	Configuration File	8
7.4	Triển khai thuật toán	9
7.4.1	Gửi message	9
7.4.2	Nhận và xử lý message	9
7.4.3	Kiểm tra điều kiện delivery	10
7.4.4	Deliver message	10
8	Logging System	10
8.1	Log Format	10
8.2	Log Example	11
9	Hiển thị và giao diện	11
9.1	Console Output	11
9.2	Interactive Commands	11
10	Verification và Testing	11
10.1	Test Suite	11
10.2	Verification Methods	12
10.2.1	Message Counting	12
10.2.2	Causal Ordering Check	12

11 Hướng dẫn sử dụng	12
11.1 Cài đặt	12
11.2 Chạy chương trình	12
11.2.1 Chạy tất cả 15 processes	12
11.2.2 Chạy từng process riêng lẻ	13
 III Tài liệu tham khảo	 14

Phần I

Giới thiệu nhóm và tiến độ công việc

1 Thông tin sinh viên

STT	Họ và tên	MSSV	Email
1	Trần Hùng Anh	22120016	22120016@student.hcmus.edu.vn

Bảng 1: Thông tin thành viên

2 Khó khăn và giải pháp

2.1 Thread Synchronization

- **Vấn đề:** Race conditions khi nhiều threads truy cập vector clock và buffer đồng thời
- **Giải pháp:** Sử dụng `threading.Lock` để đảm bảo thread-safe operations
- **Kết quả:** Không có race conditions hoặc data corruption trong quá trình chạy

2.2 Message Ordering

- **Vấn đề:** Messages đến không theo thứ tự do network delay ngẫu nhiên
- **Giải pháp:** Implement buffer với mechanism check điều kiện delivery liên tục
- **Kết quả:** 100% messages được deliver theo đúng thứ tự nhận quả

2.3 Debugging với Multiple Processes

- **Vấn đề:** Khó debug và trace issues với 15 processes chạy đồng thời
- **Giải pháp:** Implement chi tiết logging system với timestamps và structured format
- **Kết quả:** Dễ dàng trace và debug issues thông qua log files

3 Link tài liệu

- **Source code:** https://github.com/TranRoger/HTPT_LAB01
- **Video demo:** https://drive.google.com/drive/folders/1j2Y7aWs0-zsBJa_4Z3054C6_tvlxS4XF?usp=sharing

Phần II

Báo cáo bài tập

4 Tổng quan đề án

4.1 Mục tiêu

Đề án này nhằm cài đặt và minh họa thuật toán **SES (Schiper-Eggli-Sandoz)** để đảm bảo thứ tự nhân quả (causal ordering) trong hệ thống phân tán. Chương trình bao gồm 15 processes, mỗi process tự động gửi 150 messages đến mỗi process còn lại với thời gian phát sinh ngẫu nhiên.

4.2 Yêu cầu đề bài

1. 15 processes chạy đồng thời, có thể trên 1 hoặc nhiều máy tính
2. Mỗi process gửi 150 messages đến mỗi process khác
3. Thời gian phát sinh messages ngẫu nhiên (configurable)
4. Hiển thị rõ ràng việc buffering và delivery của messages
5. Log file chi tiết cho mỗi process
6. Chỉ ra khi message được buffer/delivery và lý do
7. Hiển thị cập nhật vector clock

5 Cơ sở lý thuyết

5.1 Thuật toán SES

Thuật toán SES (Schiper-Eggli-Sandoz) là một thuật toán đảm bảo thứ tự nhân quả trong hệ thống phân tán bằng cách sử dụng **Vector Clock**.

5.1.1 Vector Clock

Vector Clock là một cơ chế theo dõi quan hệ nhân quả giữa các sự kiện trong hệ thống phân tán. Mỗi process P_i duy trì một vector clock VC_i có kích thước bằng số processes trong hệ thống.

- $VC_i[i]$: Số sự kiện đã xảy ra tại process P_i
- $VC_i[j]$: Process P_i biết về bao nhiêu sự kiện của process P_j

5.1.2 Điều kiện delivery

Message M từ process P_i có thể được deliver tại process P_j khi và chỉ khi:

1. $TS(M)[i] = VC_j[i] + 1$ (Message tiếp theo từ sender)
2. $\forall k \neq i : TS(M)[k] \leq VC_j[k]$ (Không có dependencies thiếu)

Trong đó:

- $TS(M)$: Timestamp (vector clock) của message M
- VC_j : Vector clock hiện tại của process P_j

5.1.3 Thuật toán

Input: Process P_i muốn gửi message
Output: Message được gửi với timestamp
begin
 $VC_i[i] \leftarrow VC_i[i] + 1;$
 $TS \leftarrow \text{Copy}(VC_i);$
 Create message M with timestamp TS ;
 Send M to destination;
end

Algorithm 1: SES Algorithm - Send Message

Input: Process P_j nhận message M từ P_i
Output: Message được deliver hoặc buffer
begin
 if $\text{can_deliver}(M)$ **then**
 Deliver(M);
 $\forall k \neq j : VC_j[k] \leftarrow \max(VC_j[k], TS(M)[k]);$
 $VC_j[i] \leftarrow VC_j[i] + 1;$
 Try_deliver_buffered();
 else
 Buffer(M);
 end
end

Algorithm 2: SES Algorithm - Receive and Deliver Message

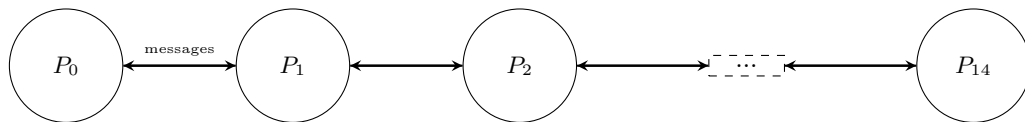
Function $\text{can_deliver}(\text{message } M \text{ from } P_i \text{ to } P_j)$:
 if $TS(M)[i] \neq VC_j[i] + 1$ **then**
 return False;
 end
 foreach $k \in \{0, 1, \dots, n-1\} \setminus \{i\}$ **do**
 if $TS(M)[k] > VC_j[k]$ **then**
 return False;
 end
 end
 return True;

Algorithm 3: Check if Message Can Be Delivered

6 Thiết kế hệ thống

6.1 Kiến trúc tổng quan

Hệ thống được thiết kế theo mô hình phân tán với 15 processes độc lập, mỗi process đóng vai trò vừa là client vừa là server.

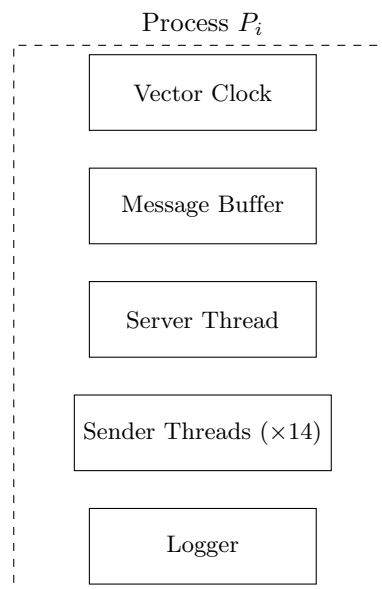


Hình 1: Kiến trúc hệ thống phân tán 15 processes

6.2 Cấu trúc Process

Mỗi process bao gồm các thành phần:

- **Vector Clock:** Theo dõi quan hệ nhân quả
- **Message Buffer:** Lưu trữ messages chưa thể deliver
- **Server Thread:** Lắng nghe incoming connections
- **Sender Threads:** 14 threads gửi messages đến các process khác
- **Logger:** Ghi log chi tiết mọi sự kiện



Hình 2: Cấu trúc của một process

6.3 Cấu trúc dữ liệu

6.3.1 VectorClock Class

```
1 class VectorClock:
2     - process_id: int
3     - clock: List[int] # Vector clock array
4     - lock: threading.Lock # Thread-safe
5
6     Methods:
7     + increment() # Increment own clock
8     + update(received_VC) # Update from received VC
9     + get_clock() # Get copy of current VC
```

Listing 1: Vector Clock Structure

6.3.2 Message Class

```
1 class Message:
2     - sender_id: int
3     - receiver_id: int
4     - content: str
5     - timestamp: List[int] # Vector clock timestamp
6     - seq_num: int # Sequence number
7
8     Methods:
9     + to_dict() # Serialize to JSON
10    + from_dict() # Deserialize from JSON
```

Listing 2: Message Structure

6.3.3 SESProcess Class

```
1 class SESProcess:
2     - process_id: int
3     - vector_clock: VectorClock
4     - buffer: List[Message] # Buffered messages
5     - config: dict # Network config
6     - messages_sent: dict # Counter per process
7     - messages_received: dict
8     - messages_delivered: dict
9
10    Methods:
11    + can_deliver(msg) # Check delivery condition
12    + deliver_message(msg) # Deliver and update VC
13    + handle_received_message(msg) # Receive handler
14    + send_message(receiver, content)
15    + try_deliver_buffered() # Check buffer
```

Listing 3: SES Process Structure

7 Cài đặt

7.1 Ngôn ngữ và công cụ

- **Ngôn ngữ:** Python 3.7+
- **Thư viện:** Chỉ sử dụng thư viện chuẩn (socket, threading, json, logging)
- **Communication:** TCP Socket
- **Concurrency:** Multi-threading
- **Configuration:** JSON

7.2 Cấu trúc dự án

```
1 HTPT_LAB01/  
2 |-- config/  
3 |   |-- config.json          # 15 processes config  
4 |-- src/  
5 |   |-- ses_process.py      # Core SES algorithm  
6 |   |-- launch_all.py       # Launch 15 processes  
7 |   |-- log_analyzer.py     # Log analysis tool  
8 |-- logs/                   # Log files directory  
9 |-- docs/                   # Documentation  
10 |-- Makefile                # Build automation  
11 |-- test.sh                 # Test suite  
12 |-- README.md               # Full documentation
```

Listing 4: Project Structure

7.3 Configuration File

File config/config.json chứa thông tin network của 15 processes:

```
1 {  
2   "processes": [  
3     {"id": 0, "host": "127.0.0.1", "port": 5000},  
4     {"id": 1, "host": "127.0.0.1", "port": 5001},  
5     ...  
6     {"id": 14, "host": "127.0.0.1", "port": 5014}  
7   ]  
8 }
```

Listing 5: Configuration Example

7.4 Triển khai thuật toán

7.4.1 Gửi message

```
1 def send_message(self, receiver_id, content):
2     # Increment vector clock
3     self.vector_clock.increment()
4
5     # Create message with current timestamp
6     msg = Message(
7         sender_id=self.process_id,
8         receiver_id=receiver_id,
9         content=content,
10        timestamp=self.vector_clock.get_clock(),
11        seq_num=self.messages_sent[receiver_id] + 1
12    )
13
14    # Send via TCP socket
15    receiver_config = self.config['processes'][receiver_id]
16    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
17    sock.connect((receiver_config['host'],
18                 receiver_config['port']))
19    sock.sendall(json.dumps(msg.to_dict()).encode())
20    sock.close()
21
22    # Log
23    self.logger.debug(f"SENT: {content} to P{receiver_id}")
```

Listing 6: Send Message Implementation

7.4.2 Nhận và xử lý message

```
1
2 def handle_received_message(self, msg):
3     self.logger.info(f"RECEIVED: {msg.content}")
4
5     # Check if can deliver immediately
6     if self.can_deliver(msg):
7         self.deliver_message(msg)
8     else:
9         # Buffer the message
10        with self.buffer_lock:
11            self.buffer.append(msg)
12            self.logger.warning(f"BUFFERED: {msg.content}")
13            self._print_buffer_reason(msg)
14
15    # Try to deliver buffered messages
16    self.try_deliver_buffered()
```

Listing 7: Receive Message Implementation

7.4.3 Kiểm tra điều kiện delivery

```
1
2 def can_deliver(self, msg):
3     current_vc = self.vector_clock.get_clock()
4     msg_ts = msg.timestamp
5     sender = msg.sender_id
6
7     # Condition 1: Next message from sender
8     if msg_ts[sender] != current_vc[sender] + 1:
9         return False
10
11    # Condition 2: No missing dependencies
12    for k in range(self.num_processes):
13        if k != sender:
14            if msg_ts[k] > current_vc[k]:
15                return False
16
17    return True
```

Listing 8: Check Delivery Condition

7.4.4 Deliver message

```
1
2 def deliver_message(self, msg):
3     # Update vector clock
4     self.vector_clock.update(msg.timestamp)
5     self.vector_clock.clock[msg.sender_id] += 1
6
7     # Add to delivered messages
8     with self.delivered_lock:
9         self.delivered_messages.append(msg)
10        self.messages_delivered[msg.sender_id] += 1
11
12    # Log delivery
13    self.logger.info(f"DELIVERED: {msg.content}")
14    self.logger.info(f" Message VC: {msg.timestamp}")
15    self.logger.info(f" Updated VC: {self.vector_clock}")
```

Listing 9: Deliver Message Implementation

8 Logging System

8.1 Log Format

Mỗi log entry bao gồm:

- Timestamp chính xác đến microsecond
- Process ID
- Log level (DEBUG, INFO, WARNING, ERROR)
- Chi tiết message và vector clock

- Lý do buffering (nếu có)

8.2 Log Example

```
1 [2025-10-17 10:30:45.123456] [P0] [INFO] SENT:
2   Message 1 to P1, VC: [1,0,0,...]
3 [2025-10-17 10:30:45.234567] [P1] [INFO] RECEIVED:
4   Message 1 from P0
5 [2025-10-17 10:30:45.234890] [P1] [INFO] DELIVERED:
6   Message 1 from P0
7 [2025-10-17 10:30:45.235000] [P1] [INFO]
8   Message VC: [1,0,0,...]
9 [2025-10-17 10:30:45.235100] [P1] [INFO]
10  Updated VC: [1,1,0,...]
11 [2025-10-17 10:30:45.345678] [P1] [WARNING] BUFFERED:
12   Message 3 from P2
13 [2025-10-17 10:30:45.345890] [P1] [WARNING]
14   Reason: Waiting for message #1 from P2 (got #3)
```

Listing 10: Log File Example

9 Hiển thị và giao diện

9.1 Console Output

Chương trình sử dụng ANSI color codes để hiển thị trạng thái:

- **Màu xanh:** Message được delivered thành công
- **Màu vàng:** Message bị buffered
- **Màu đỏ:** Lỗi hoặc exceptions

9.2 Interactive Commands

Trong khi chạy, user có thể:

- Nhấn 's': Hiển thị statistics hiện tại
- Nhấn 'q': Thoát chương trình gracefully

10 Verification và Testing

10.1 Test Suite

Dự án bao gồm test suite tự động (`test.sh`) kiểm tra:

1. Python version và syntax
2. Cấu trúc thư mục và files
3. Configuration file validation
4. Port availability
5. Integration test với 3 processes
6. Log analyzer functionality

10.2 Verification Methods

10.2.1 Message Counting

```
1
2 for i in {0..14}; do
3     delivered=$(grep -c "DELIVERED" logs/process_${i}.log)
4     echo "Process $i: $delivered messages"
5 done
6 # Expected: 2100 for each process
```

Listing 11: Verify Message Count

10.2.2 Causal Ordering Check

Kiểm tra từ log files:

- Messages được buffer khi điều kiện vi phạm
- Lý do buffer rõ ràng (missing dependencies)
- Messages được unbuffer đúng thứ tự
- Vector clock update chính xác

11 Hướng dẫn sử dụng

11.1 Cài đặt

```
1
2 cd HTPPT_LAB01
3 make setup
```

Listing 12: Setup

11.2 Chạy chương trình

11.2.1 Chạy tất cả 15 processes

```
1
2 make run
3 # Or with custom parameters
4 python3 src/launch_all.py 150 100
```

Listing 13: Run All Processes

11.2.2 Chạy từng process riêng lẻ

```
1  
2 python3 src/ses_process.py 0 150 100  
3 # Process ID: 0, Messages: 150, Rate: 100/min
```

Listing 14: Run Single Process

Phần III

Tài liệu tham khảo

1. Schiper, A., Eggli, J., & Sandoz, A. (1989). *A New Algorithm to Implement Causal Ordering*. In Proceedings of the 3rd International Workshop on Distributed Algorithms (WDAG '89), Springer-Verlag, pp. 219-232.